

FRONTIERS IN COMPUTING: DISCUSSION 5

RENTO SAIJO

2026-10-04

Processors for different kinds of work

- Computing tasks demand different combinations of flexibility, throughput, and efficiency, which explains why systems use several kinds of processors. The central processing unit (CPU) handles varied instructions, decisions, and coordination: it can run an application, respond to a web request, or direct other hardware. It also performs parallel work, especially across multiple cores, but a graphics processing unit (GPU) devotes more of its hardware to applying similar calculations to many data items at once. That arrangement suits the pixels and geometric shapes in a game frame as well as portions of scientific computing and machine learning.
- Further specialization follows the same principle. Google's tensor processing unit (TPU) targets the matrix operations common in suitable machine-learning workloads, whereas a neural processing unit (NPU) can run features such as image enhancement or speech recognition efficiently on a phone. In a data center, a data processing unit (DPU) can offload networking, storage, security, and data movement from application CPUs. A quantum processing unit (QPU) uses qubits and quantum algorithms to investigate particular classes of problems; it is no general replacement for a classical processor. One system can therefore combine several kinds of processing instead of choosing a single winner.

A photograph through the smartphone

- The smartphone gives us a compact example of cooperation. Its system on a chip (SoC) brings CPU cores, a GPU, an image signal processor (ISP), display and media engines, communication hardware, controllers, and sometimes an NPU onto a connected platform. An on-chip network moves information among these blocks, while controllers manage access to memory. Working data resides in dynamic random-access memory (DRAM), and a saved photo resides in flash storage; neither should be confused with a processing unit inside the SoC. This organization lets specialized blocks handle frequent tasks without asking the CPU to perform every calculation itself.
- When we take a photo, light reaches the image sensor, whose measurements are converted into digital values. The resulting data travel into working memory and through the ISP, which can reconstruct color from the sensor's color-filter pattern, reduce noise, and adjust color and tone. Processing occurs as data move through the system; the entire image need not fit in a small on-chip cache. Meanwhile, the CPU runs the camera application and coordinates the sequence. If the phone uses learned image enhancement, an NPU may assist, although the exact processing path varies by device and feature.
- To show the preview, the GPU can combine the camera image with the application's controls, and the display engine sends the composed result to the screen. Saving the picture adds encoding and a write to flash; sharing it invokes the modem and networking path. These stages explain why a smartphone contains many engines even though the user experiences one quick tap. Power management matters too: the phone can assign work to more efficient CPU cores or dedicated accelerators and change operating speed as demand rises or falls, extending battery life without changing the photograph's purpose.

Inside the CPU

- Inside a CPU, transistors act as controlled switches; arranged into logic gates and circuits, they can add values, make comparisons, and hold bits in registers. A running program gives those circuits a sequence of instructions. The CPU fetches an instruction, decodes what operation it requests, obtains operands, executes the operation, and makes the result available for later work. To add two values, for example, the processor may load them into registers, use an arithmetic unit to add them, and retain or store the sum. The control logic decides which units participate, while a program counter tracks the next instruction.
- Fetching data is often as important as arithmetic. Registers are closest to the executing units; small, fast caches keep recently used instructions and data nearby; larger DRAM holds more of the active program at a greater access cost. A cache miss sends the processor farther down this hierarchy. In our photo example, software may repeatedly inspect nearby image values, so reusing data already in cache can save trips to memory. The hierarchy also explains why a fast calculation does not guarantee a fast program: if the required inputs arrive slowly, arithmetic hardware waits.
- Modern CPUs overlap stages of several instructions in a pipeline, much as different stations work on successive items at once. A branch prediction lets the pipeline continue before a conditional choice is resolved; a wrong guess forces work on the incorrect path to be discarded. Separate cores execute independent instruction streams, and vector instructions apply one operation to several values within a core. Simultaneous multithreading lets one core keep instructions from more than one software thread in flight, improving use of shared execution resources when possible. Those logical threads do not create additional physical cores, and the gain depends on the program.

Matching CPUs, GPUs, and TPUs to work

- A web request shows the CPU's strength: read the request, check identity, look up information, apply rules, and choose a response. Each step can lead to another path. In contrast, rendering many image elements or repeating the same numerical operation over a large data set exposes parallel work that a GPU can process at high throughput. Both processors can often perform the task, but their organization changes how efficiently they do it. The CPU also remains valuable when it prepares data, launches GPU work, and handles decisions around that work.
- Machine learning adds another comparison. A matrix is a grid of numbers, and multiplying matrices repeats many multiply-and-add operations. A color image can be represented as a tensor with height, width, and color channels; a model may transform such numerical arrays through many layers. GPUs handle a broad range of this parallel arithmetic, while Google's TPUs devote specialized hardware to matrix-heavy neural-network computations. A TPU can be advantageous when the model, data flow, and software fit that design, yet its specialization does not make it a better graphics renderer or a universal replacement for CPUs and GPUs.

How the GPU keeps many calculations moving

- A graphics card is more than its GPU chip. It also includes video memory (VRAM), memory connections, power delivery, and cooling. Within the GPU, groups of execution units called streaming multiprocessors schedule large numbers of threads. In NVIDIA's CUDA model, threads are grouped into warps and follow a single-instruction, multiple-threads (SIMT) pattern: they run the same program while working on different data. A game scene offers a natural example, because many vertices can be transformed independently. This differs from asking one CPU instruction stream to calculate every vertex in sequence.
- SIMT has limits that matter in practice. Threads in one warp are most efficient when they take similar paths; if some take one branch and others another, the GPU executes the paths with inactive lanes masked, reducing useful work during those steps. Data must also reach the arithmetic units. VRAM supplies large scenes and textures, while on-chip registers, shared memory, and caches help reuse data locally. Insufficient memory bandwidth or poorly arranged accesses can leave many execution units idle. Specialized tensor units accelerate suitable matrix operations, and ray-tracing hardware can help with certain lighting calculations, but neither removes the need to move and organize data well.

From a 3D scene to a game frame

- Consider a locomotive in a 3D game. Its surface is represented by meshes of triangles, with vertices that describe geometry in the object's own coordinates. A vertex stage uses transformations to place the locomotive in the world, account for the camera's position, and project it onto the screen. Matrix calculations make the same sequence applicable to many vertices, which connects graphics rendering to the GPU's parallel design. The transformed vertices define where triangles could appear in the image; they do not yet specify the final color of each screen pixel.
- Rasterization determines which screen locations a projected triangle covers and generates fragments for further processing. When surfaces overlap, a depth or *z*-buffer helps retain the nearest visible surface at each location. A fragment stage then combines material properties and textures with lighting. A surface facing a light receives a different contribution from one turned away, so the angle between its normal and the light direction helps determine brightness. Interpolating normals across a triangle can make a curved object look smooth even though the underlying mesh remains polygonal.
- Aliasing makes diagonal edges appear jagged when continuous geometry is sampled onto a finite pixel grid. Anti-aliasing methods reduce that effect, sometimes by evaluating more than one sample per displayed pixel. Some games also trace selected light paths for reflections or shadows, while others render at a lower resolution and upscale the result. These are choices within a broader pipeline, with different costs and visual benefits. As the camera or scene changes, the stages run again to produce the next frame. From the photo in our phone to this game scene, a responsive image depends on coordinating computation, memory movement, and specialized hardware.